The mini coding agent: a stage-by-stage workbook

Build a coding-agent harness from an empty folder. One runnable stage at a time.

CONTENTS

What is in this manual

Preface 3

The plan at a glance 5

Before you start 6

1. What an agent is 8

2. Why the build order is unusual 9

3. The shape of the finished harness 10

Stage 0 — Scaffold 11

Stage 1 — Terminal language (ui.js) 14

Stage 2 — Config and the model client 16

Stage 3 — The simplest chat loop 19

Stage 4 — Conversation history 21

Stage 5 — The system prompt 23

Stage 6 — Path policy (workspace.js) 25

Stage 7 — Read-only tools 27

Stage 8 — Sandbox and write tools 30

Stage 9 — Command execution (run_command) 33

Stage 10 — Long-term memory 36

Stage 11 — Lifecycle hooks 39

Stage 12 — The ReAct loop (agent.js) 42

Stage 13 — Offline demo 45

Stage 14 — Sessions, trimming and CLI polish 48

Stage 15 — The full test suite 50

Stage 16 — Documentation and final acceptance 52

The course checklist 55

Where to go next 56

Troubleshooting by symptom 57

Glossary and reference 58

Page numbers refer to the folio at the foot of each page.

Preface

This is a build book, not a reading book. You will create a real program, run it after every chapter, and end with a small coding agent you can explain, test and extend.

The program is a *harness*: the code around a language model that turns text into action. It is not the model. It is the configuration, the conversation, the tools, the safety policy, the memory, the hooks and the loop. Those are the parts you can build, inspect and reason about, and they are the parts this book teaches.

Who this book is for

You should be comfortable reading modern JavaScript: modules, `async` / `await`, `fetch`, classes and basic Node APIs. You do not need to have built an agent before. You do not need a framework, a database, a cloud account or an agent SDK. You need Node.js 20 or newer, a terminal, and an OpenRouter API key for the live chapters.

The book is written for three kinds of reader:

- **The self-paced builder** who wants a working agent and an understanding of every layer.
- **The student** following a course, who needs a sequence of concrete stages and checkpoints.
- **The teacher** who wants a workbook that can be assigned stage by stage.

What you will build

By the final chapter you will have a command-line coding agent that can:

- load configuration with visible provenance;
- hold a multi-turn conversation;
- call eight tools: list, read, create, write, edit, run commands, remember and recall;
- refuse writes outside a workspace, even through symlinks and shell commands;
- run shell commands under a layered policy;
- keep long-term memory in a file;
- expose nine lifecycle hooks that can observe, rewrite or block a run;
- run a testable ReAct loop with a step limit and a truncation guard;
- prove its own safety guards offline with a scripted demo;
- pass an offline test suite and a twenty-item acceptance checklist.

What this book is not

It is not a guide to prompt tricks, model training or building a commercial product. It deliberately uses one provider and one runtime dependency so that every layer stays visible. It is also **not a security boundary**. The sandbox is a teaching policy. It is not a container, a virtual machine or an operating-system jail. The book says so repeatedly because overclaiming safety is worse than naming limits.

How this book relates to the manual and the supporting code

This workbook is the primary path. It gives you the plan, the build steps, the commands and the acceptance checks for each stage.

Two supporting documents sit beside it:

- `course-manual.md` — the theory companion. Each stage here points to the matching manual chapter. Read it when you want the *why*: the trade-offs, failure modes, protocol details and design vocabulary.
- `plan.md` — the concise build contract. Each stage here points to its section. Read it before you type if you want the exact surface in one place.
- `../mini-coding-agent-js/` — the finished implementation. Use it as a reference when you are stuck or want to compare. Do not copy whole files into your project; type the stage yourself. The learning is in the reconstruction, and the finished code contains later-stage details that would make earlier stages confusing.

Every stage in this book is incremental. You add one capability, run it, prove it, and move on. The program should work at the end of every chapter.

How to use this book

For every stage, follow the same loop:

1. **Read the plan.** Understand what you are adding and why it comes now.
2. **Build it.** Create or modify the named files. Type the code; do not paste the finished implementation.
3. **Run the incremental stage.** Use the commands in the chapter. The program should do something new.
4. **Prove it.** Tick every acceptance box. If a box fails, fix it before moving on.
5. **Reflect.** Answer the checkpoint questions, then read the matching manual chapter for the deeper explanation.

A stage is done when the program does something new **and** every acceptance box is ticked. "It compiled" is not a stage.

The promise

After this book, every coding agent you meet will look like a variation on a program you can read. You will recognise the loop, the tools, the context budget, the safety layers and the extension points. You will also know where the design stops being safe and where a production system would need a real boundary.

The plan at a glance

STAGE	PHASE	MAIN FILES	WHAT WORKS AFTER THIS STAGE	TIME
0	A	`package.json`, `.gitignore`	Project installs; one dependency	20 min
1	A	`src/ui.js`	Styled terminal output	60–90 min
2	A	`src/config.js`, `src/llm.js`	Config with provenance; OpenRouter call	2.5–3.5 h
3	A	`src/cli.js`	**Chat with the model**	60–90 min
4	A	`src/cli.js`	Multi-turn conversation	45–60 min
5	A	`src/prompt.js`	Behavioural rules in a system prompt	45–60 min
6	B	`src/workspace.js`	Writes fenced; reads open	2–3 h
7	B	`src/tools.js`	Model lists and reads files	2–3 h
8	B	`src/sandbox.js`, `src/tools.js`	Model creates, writes and edits safely	3–4 h
9	B	`src/sandbox.js`, `src/tools.js`	Shell commands with guardrails	4–5 h
10	B	`src/memory.js`	Memory across runs	3–4 h
11	C	`src/hooks.js`	Observe, rewrite, block lifecycle events	3–4 h
12	D	`src/agent.js`, `src/heartbeat.js`	Testable ReAct loop	4–6 h
13	D	`src/demo.js`	Offline proof of the guards	2–3 h
14	D	`src/cli.js`	Sessions, trimming, banner, exit codes	3–4 h
15	E	`test/*.test.js`	Full offline test suite	6–10 h
16	E	`README.md`, `demo_prompt.md`	Documentation and final acceptance	3–4 h

The times are for a focused reader typing from scratch. In a class, double them. The value is in the mistakes, not the typing.

Before you start

Prerequisites

- **Node.js 20 or newer.** The project uses ES modules, `AbortSignal.any`, `import.meta.dirname` and `node:test`.
- **One runtime dependency:** `chalk` **v5 or newer.** No framework, SDK, database or ORM.
- **An OpenRouter API key** for live mode only. The tests, the demo, `--help`, `--hooks` and `--memory` need no key and no network.
- **A terminal at least 100×30** for the banner and scorecard. Windows `cmd.exe`, PowerShell, Git Bash and a Unix shell are all usable. `run_command` inherits whichever shell launches the CLI.

Create the project

You will build in a new folder. The book uses `mini-coding-agent-js` throughout; the finished reference implementation has the same name.

```
mkdir mini-coding-agent-js
cd mini-coding-agent-js
```

Stage 0 gives you the exact `package.json`, `.gitignore` and folder layout. Do not run `npm init` blindly and accept every default; you will change most of them.

Get an API key

Create an OpenRouter key at https://openrouter.ai/keys. Put it in a local `.env` file, never in source control:

```
OPENROUTER_API_KEY=sk-or-...
```

The first live test is Stage 3. Everything before that is offline.

The supporting code

The finished implementation lives in `../mini-coding-agent-js/src/`. Use it as a reference, not a destination. It is organised by the same stages:

STAGE	REFERENCE FILES
0	`package.json`, `.gitignore`
1	`src/ui.js`
2	`src/config.js`, `src/llm.js`
3–5	`src/cli.js` (early form), `src/prompt.js`
6	`src/workspace.js`
7–9	`src/tools.js`, `src/sandbox.js`
10	`src/memory.js`, `src/memory.md`
11	`src/hooks.js`

STAGE	REFERENCE FILES
12	`src/agent.js`, `src/heartbeat.js`
13	`src/demo.js`
14	`src/cli.js` (final form)
15	`test/*.test.js`
16	`README.md`, `demo_prompt.md`

If your code disagrees with the reference, first ask *which stage am I in?* The reference contains the end state. Earlier stages intentionally lack later features.

Conventions used in this book

- Commands are shown from the project root, `mini-coding-agent-js/`.
- `$` is a shell prompt, not part of the command.
- File paths are relative to the project root unless stated otherwise.
- **Checkboxes are acceptance tests.** Tick them by running the command, not by reading the code.
- `TODO` in a code sketch means "you write this line".
- The words *stage*, *chapter* and *step* are used interchangeably; every stage is one chapter.

A note on safety

The harness in this book is designed to make unsafe actions **visible and unlikely**, not impossible. The workspace fence stops accidental writes. The command scanner catches common destructive command lines. The environment allow-list keeps the API key out of child processes. None of that survives a determined attacker who can run arbitrary code. The book returns to this in Stage 9 and Stage 16. Read the limits before you point the agent at anything you care about.

Part I: Orientation

1. What an agent is

An agent is a loop around a model:

1. The harness sends a conversation and a list of tools.
2. The model replies with text and possibly a tool call.
3. The harness runs the tool and appends the result as a message.
4. The loop repeats until the model answers without calling a tool, or a limit is reached.

That is ReAct: **Re**ason, **Act**, observe. You do not need a framework. You need an array, a `for` loop and a `fetch`.

The important consequence: the model has no memory, no filesystem and no shell. Every capability is a tool the harness exposes, and every observation is a string the harness writes back. The safety of the system lives in the harness, not in the model.

For the full mental model, read `course-manual.md` Part 0.

2. Why the build order is unusual

A traditional engineering plan would build the sandbox, then the tools, then the model client, and only then let the program talk. That is easier to diagram and much harder to learn from, because nothing visible happens until late.

This book builds in the opposite direction:

1. **Make it talk.** Config, HTTP, one model call. You have a chatbot in Stage 3.
2. **Give it memory of the conversation.** Stage 4.
3. **Give it a job description.** Stage 5.
4. **Give it hands.** Stages 6–9, one capability at a time.
5. **Give it persistence and observability.** Stages 10–11.
6. **Formalise the loop.** Stage 12.
7. **Prove it.** Stages 13–16.

Every stage produces a runnable program that is immediately more capable than the last. You feel the feature, and you meet the failure modes in the order a real user meets them.

3. The shape of the finished harness

```
mini-coding-agent-js/
  package.json          type: module, one dependency
  .gitignore            node_modules, .env, workspace/*
  .env                  your key (never committed)
  .env.example          the shape of the configuration
  src/
    cli.js              the program: flags, sessions, banner, exit codes
    config.js           .env, precedence, provenance, bounds
    llm.js              one POST to OpenRouter
    ui.js               terminal language; returns strings
    prompt.js           the system prompt, rebuilt per task
    workspace.js        path policy: reads open, writes fenced
    sandbox.js          write and command policy
    tools.js            schemas, validation, dispatch, eight tools
    memory.js           file-backed long-term memory
    hooks.js            nine lifecycle events
    agent.js            the ReAct loop as a library
    heartbeat.js        "still waiting" timer
    demo.js             scripted model for offline proof
    memory.md           the memory store itself
  test/                 one suite per module
  workspace/            the only folder the agent may write to
```

The import graph points one way: `cli.js` → `agent.js` → the modules. `agent.js` never imports `cli.js`. `ui.js` is a leaf. That shape is what makes the loop testable.

Part II: The stages

> **How each chapter is organised.** Every stage has a **Plan** (what you are adding and in what order), a **Build** section (files and concrete instructions), a **Run the incremental stage** section (the commands that prove the new capability), a **Prove it** checklist, and a **Checkpoint**. The full contract for each stage is in `plan.md`; the deeper explanation is in the matching `course-manual.md` chapter.

Stage 0 — Scaffold

What you can do at the end: install the project and confirm that it has exactly one runtime dependency.

Plan: `plan.md` § Stage 0 · `course-manual.md` § Stage 0

Plan

1. Create the project folder and the flat layout: `src/`, `test/`, `workspace/`.
2. Write a `package.json` that declares ES modules, Node 20+, one dependency and the scripts you will use throughout the book.
3. Write a `.gitignore` that protects `node_modules`, `.env` and generated workspace files.
4. Install `chalk` and prove it is the only runtime dependency.

Two constraints chosen here shape every later stage: **one runtime dependency** and **one writable folder**. The dependency rule keeps the model call, timeout and filesystem visible. The workspace rule gives the safety policy somewhere to point.

Build

Files to create

- `package.json`
- `.gitignore`
- `.env.example`
- `src/.gitkeep`
- `workspace/.gitkeep`

Create the folders:

```
mkdir -p src test workspace
touch src/.gitkeep workspace/.gitkeep
```

Write `package.json`:

```
{
  "name": "mini-coding-js",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "A tiny Node coding-agent harness: ReAct loop + 8 tools, one dependency for colour.",
  "scripts": {
    "start": "node src/cli.js",
    "demo": "node src/cli.js --demo",
    "hooks": "node src/cli.js --hooks",
    "memory": "node src/cli.js --memory",
    "test": "node --test"
  },
  "engines": { "node": ">=20" },
  "dependencies": { "chalk": "^5.6.2" }
}
```

Write `.gitignore`:

```
node_modules/
.env
workspace/*
!workspace/.gitkeep
*.log
.DS_Store
```

Write `.env.example`:

```
# Copy this file to .env and fill in your key.
OPENROUTER_API_KEY=sk-or-...
# Optional overrides:
# OPENROUTER_MODEL=deepseek/deepseek-v4.1-flash-1017
# CMD_POLICY=ask
# MAX_STEPS=12
```

Install the dependency:

```
npm install
```

Do not add `commander`, `dotenv`, an HTTP client or an agent SDK. Stage 2 parses `.env` by hand and calls `fetch` directly.

Run the incremental stage

```
cd mini-coding-agent-js
npm install
npm ls --depth=0
node --test
node --version
```

Expected: `npm ls` shows `chalk` and nothing else under `dependencies`; `node --test` reports zero tests without failing; `node --version` is 20 or newer.

Prove it

- [ ] `npm install` installs exactly `chalk`.
- [ ] `npm ls --depth=0` shows `chalk` and nothing else in `dependencies`.
- [ ] `node --test` reports `0 tests` without failing.
- [ ] `workspace/` exists and is git-ignored except `.gitkeep`.
- [ ] `"type": "module"` is set.
- [ ] `node --version` is 20 or newer.
- [ ] `.env` is ignored by git.

If it goes wrong

- `Cannot use import statement outside a module` — you forgot `"type": "module"` or you are running an old Node.

- `npm ls` **shows extra packages** — remove them with `npm uninstall <name>` and re-run the audit. Do not keep a package "for later".
- `workspace/` **disappears from git** — commit `workspace/.gitkeep`, not the folder itself.

Checkpoint

1. Why does the book insist on a single runtime dependency?
2. What is the difference between where the process starts and where the agent may write?
3. Why is `workspace/.gitkeep` committed while the rest of `workspace/` is ignored?

Answers and fuller discussion: `course-manual.md` § Stage 0.

Stage 1 — Terminal language (`ui.js`)

What you can do at the end: print styled, classified terminal output through helpers that return strings.

Plan: `plan.md` § Stage 1 · `course-manual.md` § Stage 1

Plan

1. Create a small palette around `chalk`.
2. Make every helper **return a string**. Nothing in `ui.js` prints.
3. Derive result colour from the observation text, not from a boolean passed by the caller.
4. Add `explainExit(code)` so non-zero exit codes have plain-English meaning.
5. Add one loud, greppable `forbidden` helper.
6. Test that model text is printed verbatim and that colour can be turned off.

Build

Files to create

- `src/ui.js`
- `test/ui.test.js`

`src/ui.js` should export helpers with this surface:

```
title, field, kv, step, think, tool, result, forbidden, pass, fail, rule,
hookTitle, hookLine, hookField, hookLive, hookNote, hookTrace, hookFlow,
memory, exitNote, answerRule, answer, warn, waiting, error, note, prompt, dim
```

The rules:

- Every helper returns a string. The caller decides where it goes.
- `ui.result(text)` is red when the first line starts with `ERROR`, `DENIED` or `FORBIDDEN`, or contains a non-zero `exit code N`; green otherwise.
- `explainExit(code)` maps `0`, `1`, `2`, `3`, `126`, `127`, `128–164` and Windows `9009` to plain English.
- `ui.forbidden(action, requested, reason)` produces one loud line:

```
⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt
```

- Model output must pass through verbatim. Do not build a home-made tag parser that eats `<red>` or backslashes; use `chalk` around the text, not inside it.

In `test/ui.test.js`, cover: helper returns a string; ANSI on a TTY; plain text when colour is off; red/green classification; exit-code descriptions; verbatim model output.

Run the incremental stage

```
node --test test/ui.test.js
node -e "import('./src/ui.js').then(ui => console.log(ui.title('hello')))"
```

Expected: the test suite passes, and the one-liner prints a styled title to your terminal.

Prove it

- [ ] Every helper returns a string.
- [ ] Styles become ANSI on a TTY and read back as the original text when colour is off.
- [ ] A failed result is red; a clean result is green.
- [ ] Exit code 1 gets a description via `explainExit`.
- [ ] A successful command shows its first stdout line in a dim annotation.
- [ ] `forbidden` produces bold red capitals.
- [ ] Model text containing `<red>`, backslashes or JSON prints verbatim.

If it goes wrong

- `console.log` **appears inside a helper** — move printing to the caller. The loop will later inject a `log` function.
- **Colours disappear in tests** — `chalk` detects TTY; test the plain-text path or set the colour level explicitly in the test.
- **Result colour is wrong** — classify from the first line of the observation, not from an `ok` argument.

Checkpoint

1. Why should `ui.js` never call `console.log`?
2. How does `ui.result` decide between green and red?
3. Why is `FORBIDDEN` deliberately different from `ERROR`?

Answers and fuller discussion: `course-manual.md` § Stage 1.

Stage 2 — Config and the model client

What you can do at the end: load configuration with provenance and make one real OpenRouter request.

Plan: `plan.md` § Stage 2 · `course-manual.md` § Stage 2

Plan

Config first:

1. Find the nearest `.env` by walking up from the project root.
2. Parse it by hand: `KEY=VALUE`, optional `export`, comments, quotes, blank values.
3. Apply precedence: explicit overrides > nearest `.env` > shell environment > defaults.
4. Fingerprint the key without printing it.
5. Normalise a bare model name to `vendor/model` and report the change.
6. Return values **and provenance**: where the key and model came from, which env file was used, and whether an override genuinely changed anything.

Then the client:

7. Write `chat()` as one POST with a composed abort signal and timeout.
8. Treat transport and HTTP failures as throws; sanitise the assistant message before it returns.

Build

Files to create

- `src/config.js`
- `src/llm.js`
- `test/config.test.js`
- `test/llm.test.js`

`src/config.js`

- `PROJECT_ROOT = path.resolve(import.meta.dirname, '..')`.
- `findEnvFile(startDir)` walks to the filesystem root; the nearest `.env` wins.
- `loadDotEnv(file)` parses the file without a dependency.
- `fingerprint(secret)` returns `not set`; all `*` when the key is 12 characters or fewer; otherwise `first5...last4 (N chars)`.
- `normalizeModel` maps a bare name to a vendor and leaves a name containing `/` alone.
- `loadConfig(overrides, { envFile, startDir })` returns values plus `apiKeySource`, `modelSource`, `envFile`, `apiKeyOverrodeShell`, `modelNormalised`, and the bounds below.

Defaults:

```
model                deepseek/deepseek-v4.1-flash-1017
CMD_POLICY           ask
MAX_STEPS            12
REQUEST_TIMEOUT_MS   90000
WORKSPACE_DIR        workspace
MEMORY_FILE          src/memory.md
MEMORY_MAX_ENTRIES   60
MEMORY_PROMPT_CHARS  1200
```

`src/llm.js`

- `chat({ apiKey, model, messages, tools, signal, timeoutMs = 90000 })`.
- A missing key throws `OPENROUTER_API_KEY is not set...`.
- Compose the caller signal and the timeout with `AbortSignal.any([signal, AbortSignal.timeout(timeoutMs)])`.
- POST to `https://openrouter.ai/api/v1/chat/completions` with `Authorization`, `Content-Type`, and `X-Title: min-coding-js`.
- Body: `{ model, messages, tools, max_tokens: 2048 }`.
- Read the response body as text **before** parsing; check the status first.
- Give useful messages for `401` (bad/revoked key), `402` (out of credits), a timeout (`request timed out after <n>s`), and an `.error` field on a `200`.
- `sanitizeAssistant(message)` rebuilds the message with only `role`, `content` and `tool_calls`; drops `reasoning` / `reasoning_content`; deletes `content` only when it is null and tool calls exist.

Mock `fetch` in `test/llm.test.js`. Do not make a network call in a test.

Run the incremental stage

```
node --test test/config.test.js test/llm.test.js
node -e "import('./src/config.js').then(m => console.log(m.loadConfig({}).apiKeySource))"
```

Expected: tests pass offline. The one-liner prints the source of your key if `.env` exists, or `not set` if it does not.

Prove it

- [ ] `loadConfig` resolves `.env`, shell and overrides in the right order.
- [ ] `.env` beats a stale shell key; the shell still supplies names `.env` does not define.
- [ ] A missing `.env` is not an error.
- [ ] The fingerprint never exposes the whole key.
- [ ] Bare model names normalise to `vendor/model`, and the change is reported.
- [ ] An override is reported only when the value genuinely differed.
- [ ] `sanitizeAssistant` drops reasoning and keeps only the three fields.
- [ ] `llm` tests mock `fetch`; timeout, `401`, `402`, non-JSON and `.error` are all covered offline.

If it goes wrong

- **Your key is printed in the terminal or a test** — stop, rotate the key, and use `fingerprint` everywhere.
- `res.json()` **throws on an error page** — read the body as text first, then parse after the status check.
- **The timeout does not fire** — make sure you pass the composed signal to `fetch`, not only to the options.
- **A mocked** `fetch` **leaks into another test** — restore it in `finally`.

Checkpoint

1. Why does `.env` beat an exported shell variable in this project?
2. What does `fingerprint` protect against, and why include the length?
3. Why does `chat()` throw on a `401` but `runTool()` return an `ERROR` string for a bad edit?
4. What breaks if you echo `reasoning_content` back to the provider?

Answers and fuller discussion: `course-manual.md` § Stage 2.

Stage 3 — The simplest chat loop

What you can do at the end: talk to the model from the terminal with a real task.

Plan: `plan.md` § Stage 3 · `course-manual.md` § Stage 3

Plan

1. Build the smallest `cli.js` that loads config, sends one user message and prints the answer.
2. Read a task from the command line or interactively with `readline`.
3. Parse a few flags by hand: `--help`, `--model`, `--max-steps`.
4. Make exit codes meaningful: `2` for no task/no TTY, `1` for a request failure.
5. Keep `--help` working with no key and no network.

Build

Files to create

- `src/cli.js` (minimal)
- `test/input.test.js`

Implementation:

- `askTask({ input, output })`: prompt `task ›`, trim, re-ask on blank, return `null` on Ctrl-C, iterate with `for await (const raw of rl)`.
- Hand-rolled flag parsing: `--help`, `--model`, `--max-steps`; every non-flag becomes part of the task.
- Load config, build `[system, user]`, call `chat`, print `ui.answer`.
- No task and stdin is not a TTY → print a hint and exit `2`.
- A request failure → print the error and exit `1`.
- `--help` prints usage and exits `0` before any key check.

Run the incremental stage

```
node src/cli.js --help
node src/cli.js "Say hello in one sentence"
node src/cli.js < /dev/null; echo "exit=$?"
```

Expected: `--help` prints usage without a key; the live command prints a model reply; the non-TTY command exits `2` with a helpful message.

Prove it

- [ ] `node src/cli.js --help` prints usage with no key.
- [ ] `node src/cli.js "Say hello"` reaches OpenRouter and prints the reply.
- [ ] No task + non-TTY exits `2` with a helpful message.
- [ ] Missing/invalid key exits `1` with a clear message.

- [ ] `--model` overrides `.env`.

If it goes wrong

- **The program hangs** — you are waiting for stdin that will never arrive. Check the TTY before prompting.
- **Ctrl-C throws** — return `null` from `askTask` instead of letting the signal become an exception.
- `--help` **asks for a key** — move the help branch before `loadConfig` or make `--help` independent of the key.

Checkpoint

1. Why does `--help` need to work without a key?
2. Why return `null` on Ctrl-C instead of letting the process throw?
3. What does exit code `2` mean, and why is it different from `1`?

Answers and fuller discussion: `course-manual.md` § Stage 3.

Stage 4 — Conversation history

What you can do at the end: hold a multi-turn conversation in one process.

Plan: `plan.md` § Stage 4 · `course-manual.md` § Stage 4

Plan

1. Keep one `messages` array for the life of the process.
2. Append the user message, call the model, append the assistant reply.
3. Add `/quit`, `/exit` and `/new`.
4. Make `/new` clear the conversation and re-add only the system prompt.
5. Keep one-shot mode working.
6. Print a session summary on exit.

Build

Files to modify

- `src/cli.js`

Implementation:

- Start with the system message in the array.
- Interactive banner: `interactive session - type a task below (/quit exits)`.
- Commands: `/quit`, `/exit`, `/new`.
- Every turn: append `user` → model call → append `assistant`.
- `/new` resets to `[system]` and nothing else.
- One-shot mode still sends one task and exits.
- On exit, print `session over - N turn(s)`.

Run the incremental stage

```
node src/cli.js
```

Then:

```
my name is Craig
what is my name?
/new
what is my name?
/quit
```

Expected: the second question works; after `/new` the model no longer knows the name; the session summary prints the turn count.

Prove it

- [ ] A follow-up references the previous turn.
- [ ] `/new` clears the conversation.
- [ ] `/quit` and `/exit` end the session.
- [ ] The session summary prints the turn count.
- [ ] One-shot still works.

If it goes wrong

- **The model forgets immediately** — you rebuilt `messages` each turn instead of keeping one array.
- `/new` **deletes files** — it should only reset the conversation. The workspace and memory are separate lifetimes.
- **The system message appears twice after** `/new` — reset the array to exactly `[system]`.

Checkpoint

1. Why does the model "remember" the previous turn if it has no memory?
2. What exactly does `/new` clear, and what does it leave alone?
3. What is the difference in lifetime between `messages` and `workspace/`?

Answers and fuller discussion: `course-manual.md` § Stage 4.

Stage 5 — The system prompt

What you can do at the end: inject a tested, rebuildable behavioural contract into every task.

Plan: `plan.md` § Stage 5 · `course-manual.md` § Stage 5

Plan

1. Move the rules out of `cli.js` into `src/prompt.js`.
2. Write `systemRules(tools = [])` as prose, not a magic string.
3. Write `buildSystemPrompt({ memory, log })` so memory can be added later without changing call sites.
4. At this stage, pass `tools = []`; do not name tools that do not exist.
5. Rebuild the prompt for each task; keep it first in the message array.

Build

Files to create

- `src/prompt.js`
- `test/prompt.test.js`

Files to modify

- `src/cli.js`

Rules to include:

1. Look before you write. Read the file before editing it.
2. Prefer `create_file` for new files, `edit_file` for small changes, `write_file` for whole-file replacement.
3. Copy `old_string` exactly, including whitespace. The edit tool matches literally.
4. Run what you wrote. A file that has not executed is a hypothesis.
5. Do not invent tool results. The harness supplies them; wait for them.
6. Be concise. The user is watching a terminal.
7. One tool call per step. Interleaved edits are hard to audit.
8. Remember useful facts, not task progress. (Meaningful once Stage 10 exists.)

At Stage 5, `systemRules(tools = [])` should default to `[]`. Stage 7 will pass the real tool names. `buildSystemPrompt({ memory, log })` accepts `memory` and `log` but does not use them yet. The system message is first on every turn, and `/new` keeps it.

Run the incremental stage

```
node --test test/prompt.test.js
node src/cli.js "What rules are you following?"
```

Expected: the test asserts the rule text and the injection point; the live reply reflects the prompt.

Prove it

- [ ] Rules are the first `system` message on every turn.
- [ ] `/new` keeps the system prompt.
- [ ] No tool names are listed while no tools exist.
- [ ] `buildSystemPrompt` accepts `memory` and `log` without using them yet.
- [ ] `prompt.test.js` asserts the rule text and the injection point.

If it goes wrong

- **The model hallucinates tool calls** — you listed tools that do not exist yet. Pass `[]`.
- **Rules vanish after** `/new` — re-add the same system message.
- **The prompt is still in** `cli.js` — move it. The prompt is program text with its own tests.

Checkpoint

1. Why keep the model's behavioural rules out of `cli.js`?
2. Why pass an empty tool list at Stage 5 instead of listing all eight tools optimistically?
3. Why is the system prompt rebuilt for each task rather than appended to history?

Answers and fuller discussion: `course-manual.md` § Stage 5.

Stage 6 — Path policy (`workspace.js`)

What you can do at the end: resolve paths with a strict write policy and an open read policy.

Plan: `plan.md` § Stage 6 · `course-manual.md` § Stage 6

Plan

1. Create a `Workspace` around a root folder.
2. Write a strict `resolve(requested)` for writes: relative only, no escapes, no forbidden first segments, no symlink smuggling.
3. Expand `~`, `%VAR%`, `${VAR}` and `$VAR` **before** containment checks.
4. Harden symlinks by resolving the deepest existing ancestor and re-appending the not-yet-created tail.
5. Write a permissive `resolveRead(requested)`.
6. Normalise `rel` to forward slashes; root is `.`.

Build

Files to create

- `src/workspace.js`
- `test/workspace.test.js`

Implementation:

- Export `PathError`.
- `new Workspace(root)`: resolve, `mkdir -p`, store `fs.realpathSync(root)`.
- `workspace.resolve(requested)` rejects:
- empty or non-string paths;
- absolute paths;
- drive-relative paths (`C:foo`);
- UNC paths (`\\server\share`, `//server/share`);
- `~`, `$` and `%` prefixes after expansion;
- a forbidden first segment (`users`, `windows`, `program files`, `desktop`, `documents`, `downloads`, `appdata`, `onedrive`, `library`, `system32`, and similar).
- Symlink hardening: resolve the deepest existing ancestor with `fs.realpathSync`, re-append the not-yet-created tail, re-check containment.
- `workspace.resolveRead(requested)` allows absolute paths, outside paths and `~`.
- `rel` is always `/`-separated; the root is `.`.

Why `path.resolve` plus `startsWith` is not enough: `workspace-evil` starts with `workspace`, and a junction inside the workspace can point anywhere. Containment must compare resolved paths on a path boundary, and symlinks must be resolved.

Run the incremental stage

```
node --test test/workspace.test.js
```

Expected: legitimate paths resolve; every escape shape is refused; reads outside the workspace succeed; a symlinked folder cannot smuggle a write outside.

Prove it

- [ ] Legitimate relative paths resolve and `rel` is correct.
- [ ] Every escape shape in the rejection list is refused.
- [ ] Reads outside the workspace are allowed.
- [ ] A symlinked folder inside the workspace cannot smuggle a write outside.
- [ ] A missing path is still checked against its deepest existing ancestor.
- [ ] `PathError` is distinguishable from an I/O error.

If it goes wrong

- `startsWith(root)` **accepts** `workspace-evil` — compare with a separator or use `path.relative`.
- **A** `%VAR%` **path slips through** — expand before the containment check, not after.
- **A new file cannot be resolved** — `realpathSync` fails on a missing tail. Resolve the deepest existing ancestor only.
- **Windows and Unix disagree about** `/` — normalise `rel` at the boundary.

Checkpoint

1. Why are reads open but writes fenced?
2. Why must variable expansion happen before containment checks?
3. What does `realResolve` protect against that a string check does not?
4. Why is `rel` normalised to forward slashes?

Answers and fuller discussion: `course-manual.md` § Stage 6.

Stage 7 — Read-only tools

What you can do at the end: the model can list files and read numbered file contents.

Plan: `plan.md` § Stage 7 · `course-manual.md` § Stage 7

Plan

1. Define tools in the OpenAI function-calling schema shape.
2. Write `validate(name, rawArgs)` for JSON strings or objects.
3. Write `runTool(name, rawArgs, ctx)` that dispatches and always returns a string.
4. Implement `list_files` with depth and line bounds.
5. Implement `read_file` with offset, limit, numbering and an honest truncation marker.
6. Wire the manual tool loop into `cli.js`: call, dispatch, append `role: 'tool'`, call again.

Build

Files to create

- `src/tools.js` (partial: `list_files`, `read_file`)
- `test/tools.test.js` (read-only tests)

Files to modify

- `src/prompt.js` (mention the two tools)
- `src/cli.js` (tool loop)

Schema shape:

```
{
  type: 'function',
  function: {
    name,
    description,
    parameters: { type: 'object', properties, additionalProperties: false }
  }
}
```

`validate(name, rawArgs)`:

- Accept a JSON string or an object.
- Parse, type-check, integer-check, check required fields and reject unknown keys.
- Return a helpful error string when invalid.

`runTool(name, rawArgs, ctx)`:

- Dispatch to the handler.
- An unknown tool returns a helpful list of available tools.
- Handler exceptions become text observations; they do not escape.

`list_files`:

- `path` defaults to `'.'`; `depth` defaults to 3, max 6.
- Use `resolveRead`.
- Missing folder → `ERROR: no such folder: <rel>`.
- Walk recursively, sorted.
- Skip `.git`, `node_modules`, `.venv`, `__pycache__`.
- Directories print `<rel>/`; files print `<rel> (<size>b)`.
- Stop at the depth limit and 200 lines.
- End with `<dirs> folder(s), <files> file(s) under <start.rel>`.

`read_file`:

- `path` required; `offset` defaults to 1; `limit` defaults to 200, max 2000.
- Use `resolveRead`.
- Missing → `ERROR: no such file: <rel>`.
- Directory → `ERROR: <rel> is a folder - use list_files`.
- Header: `"<rel> (<total lines>)"`.
- Numbered lines right-aligned to width 4, then `|`.
- Truncation marker: `... [<n> more lines]`.
- Clamp `offset` to at least 1 and `limit` to 1..2000.

In `cli.js`, after a reply with `tool_calls`: print `ui.tool`, dispatch each call, append exactly one `role: 'tool'` message per `tool_call_id`, print `ui.result`, then call the model again.

Run the incremental stage

```
node --test test/tools.test.js
node src/cli.js "List the files in the current folder, then read package.json"
```

Expected: the model calls `list_files` and `read_file`; you see numbered contents; bad arguments return text errors rather than crashing.

Prove it

- [ ] The model can call `list_files` and see a listing.
- [ ] The model can call `read_file` and see numbered contents.
- [ ] Bad arguments return text errors, not crashes.
- [ ] Reading outside the workspace succeeds (`../package.json`).
- [ ] Missing file → `ERROR: no such file`.
- [ ] Directory read → `ERROR: ... is a folder`.
- [ ] Unknown tool name gets a helpful list.
- [ ] Every `tool_call_id` receives exactly one `tool` message.

If it goes wrong

- `arguments` **is an object and your parser expects a string** — normalise both shapes.
- **A handler throws and ends the run** — catch it and return a string.
- **The listing walks** `node_modules` — add the skip list before you test on a real project.
- **A truncated file looks complete** — the marker is not optional.

Checkpoint

1. Why must a tool handler return a string rather than throw?
2. Why is `arguments` a JSON string, and what happens if you pass an object?
3. Why does `read_file` include a total line count and a truncation marker?

Answers and fuller discussion: `course-manual.md` § Stage 7.

Stage 8 — Sandbox and write tools

What you can do at the end: the model can create, overwrite and edit files inside the workspace, and every refusal is loud.

Plan: `plan.md` § Stage 8 · `course-manual.md` § Stage 8

Plan

1. Create a `Sandbox` layer above the workspace path policy.
2. Give refusals their own error type and a single formatting point.
3. Implement `save`, `remove` and a root guard.
4. Add `create_file`, `write_file` and `edit_file` tools.
5. Make `edit_file` literal: exact match, count occurrences, refuse ambiguity.
6. Wire the sandbox into `cli.js`.

Build

Files to create

- `src/sandbox.js`
- `test/sandbox.test.js`

Files to modify

- `src/tools.js` (add `create_file`, `write_file`, `edit_file`)
- `src/cli.js` (wire the sandbox)

`src/sandbox.js`:

- `SANDBOX_DIR = path.resolve(import.meta.dirname, '..', 'workspace')`.
- `ACTIONS = { SAVE: 'save', EDIT: 'edit', DELETE: 'delete' }`.
- `ForbiddenError` carries `action`, `requested`, `reason`; `name = 'ForbiddenError'`, `code = 'FORBIDDEN'`.
- `new Sandbox({ workspace, root, log })`.
- `sandbox.check(action, requested)`:
- unknown action → `TypeError`;
- delegate to `workspace.resolve`;
- convert `PathError` to `ForbiddenError`;
- increment `refusals`;
- print exactly one `ui.forbidden`;
- throw.
- `sandbox.save(requested, content)`: check, create parents, write, return `{ existed, bytes }`.
- `sandbox.remove(requested)`: check, `rmdirSync` for empty directories, `unlinkSync` for files; missing → `{ missing: true }`; refuse deleting the root.
- `sandbox.summary()`: `"save, edit, delete - only inside <root>"`.

Tools:

- `create_file`: check; if it exists → `ERROR: already exists - use edit_file or write_file`; otherwise write and return `created <rel> (<bytes> bytes)`.
- `write_file`: check; create parents; overwrite; return `created` or `overwrote`.
- `edit_file`: `path`, `old_string`, `new_string`, `replace_all` (default false).
- Missing file → `ERROR`.
- Empty `old_string` → `ERROR`.
- Count exact occurrences.
- 0 → `ERROR: old_string not found... Read the file and copy exactly, including whitespace.`
- More than 1 without `replace_all` → `ERROR: old_string appears N times...`
- Replace literally with the callback form of `String.replace` or `split` / `join`.
- Return `edited <rel>: <n> replacement(s), first at line <line>`.

The `delete_file` note: the plan lists a delete tool for this stage, but the finished eight-tool harness keeps `sandbox.remove` and its root guard and does **not** expose a model-facing `delete_file`. Implement it if you want a ninth tool, but keep the root guard and the same `FORBIDDEN` contract, and say so in your README.

Run the incremental stage

```
node --test test/sandbox.test.js test/tools.test.js
node src/cli.js "Create hello.txt with the word hello, then edit it to say goodbye"
node src/cli.js "Save a file called escape.txt in the parent folder"
```

Expected: the first task creates and edits inside `workspace/`; the second prints one ⛔ `FORBIDDEN` line and the model receives a `FORBIDDEN:` observation.

Prove it

- [ ] `create_file` writes inside the workspace.
- [ ] `create_file` refuses to clobber.
- [ ] `write_file` overwrites.
- [ ] `edit_file` works with exact matching.
- [ ] `edit_file` treats `$1` and `$&` literally.
- [ ] `edit_file` fails on missing file, empty `old_string` and ambiguity.
- [ ] Every escape shape is refused for save/edit/delete.
- [ ] Deleting the sandbox root is refused.
- [ ] An unknown action to `check()` throws `TypeError`.
- [ ] A refused write prints exactly one `FORBIDDEN` line.

If it goes wrong

- **Refusals are formatted in three places** — route every refusal through the sandbox.
- `edit_file` **corrupts** `$&` — you used a replacement string. Use a callback.
- `create_file` **silently overwrites** — "create" must refuse; replacement belongs to `write_file`.

- **Deleting the root succeeds** — add the explicit root guard before the path check.

Checkpoint

1. What are the two questions `workspace.js` and `sandbox.js` answer?
2. Why is an unknown sandbox action a `TypeError` rather than a `ForbiddenError`?
3. Why does `create_file` refuse to overwrite while `write_file` does not?
4. Why does `edit_file` use the callback form of `String.replace`?

Answers and fuller discussion: `course-manual.md` § Stage 8.

Stage 9 — Command execution (`run_command`)

What you can do at the end: the model can run shell commands under a layered policy.

Plan: `plan.md` § Stage 9 · `course-manual.md` § Stage 9

Plan

1. Tokenise a command line well enough to see writes and quoted arguments.
2. Extract write targets from redirections, copy/move/delete commands, downloaders and PowerShell aliases.
3. Add the hard deny-list that policy cannot override.
4. Add `checkCommand` to the sandbox: write scan, then policy.
5. Implement `deny` / `allow` / `ask`, with `ask` degrading to `deny` when stdin is not a TTY.
6. Execute with a child environment allow-list, a timeout and a bounded result format.

Build

Files to create

- `test/shell-sandbox.test.js` (or extend `test/sandbox.test.js`)

Files to modify

- `src/sandbox.js` (`checkCommand`, `tokenizeCommand`, `shellWriteTargets`)
- `src/tools.js` (`run_command`)
- `src/cli.js` (`--yes` / `-y`, `--allow-commands`, `makeConfirm()`)

`tokenizeCommand(text)`:

- Split on whitespace, honouring single and double quotes.
- Emit operators as tokens: `>`, `>>`, `<`, `|`, `||`, `&`, `&&`, `;`.
- Strip the quotes.

`shellWriteTargets(command)`:

- Redirections `>` / `>>` → the following token (handle `2> file`).
- `DEST_LAST` commands (`copy`, `cp`, `move`, `mv`, `xcopy`, `ren`, `rename`, `ln`, `install`, `scp`, `rsync`) → the last positional, at least two.
- `robocopy` → positional argument 2.
- `ALL_POSITIONAL` commands (`del`, `erase`, `rd`, `rmdir`, `rm`, `md`, `mkdir`, `touch`, `tee`, `unlink`, `truncate`, `shred`) → every positional.
- Downloaders: `curl` `-o` / `--output` / `-O` / `--remote-name`; `wget` `-O` / `--output-document`.
- PowerShell: `out-file`, `set-content`, `add-content`, `new-item`, `ni`, `remove-item`, `copy-item`, `move-item`, `rename-item`, with `-destination`, `-dest`, `-path`, `-filepath`, `-literalpath`.
- Strip `.exe`, `.cmd`, `.bat`, `.com`, `.ps1`; compare case-insensitively.
- A switch is `--flag`, `-f` or a Windows `/x`; a POSIX path like `/home` is **not** a switch.

`run_command` order:

1. trim; empty → error;
2. hard deny-list;
3. `sandbox.checkCommand`;
4. policy (`deny` / `allow` / `ask`);
5. human confirmation;
6. execute.

Hard deny-list before policy: recursive delete of a root path; `del` with `/s` or `/q`; `format`; `diskpart`; `shutdown`; `reboot`; `git push --force` / `-f`; `curl` / `wget` piped into `sh` / `bash`; a fork bomb; `> /dev/sd...`.

Execution:

- `node:child_process.exec` in `workspace.root`.
- `timeout: 30000`, `maxBuffer: 1 MB`, `windowsHide: true`, `encoding: 'utf8'`.
- `safeEnv(source)` keeps only `PATH`, `PATHEXT`, `HOME`, `USERPROFILE`, `TMPDIR`, `TEMP`, `TMP`, `SYSTEMROOT`, `COMSPEC`, `SHELL`, `LANG`, `LC_ALL`, `TERM`.

Result format:

```
exit code <n>
stdout:
...
stderr:
...
```

- `(no output)` if both streams are empty.
- Clip each stream to 6,000 chars with `... [<n> chars truncated]`.
- On timeout, append `[tool] command timed out after 30s` and return a non-zero result.

Run the incremental stage

```
node --test test/shell-sandbox.test.js
node src/cli.js "Run node --version and tell me the result"
CMD_POLICY=deny node src/cli.js "Run node --version"
node src/cli.js "Copy package.json to the Desktop"
```

Expected: the first command runs; `CMD_POLICY=deny` refuses; the Desktop copy is refused **before** any approval prompt.

Prove it

- [ ] `run_command` returns exit code, stdout and stderr.
- [ ] Hard-denied commands are refused even with `--yes`.
- [ ] `CMD_POLICY=deny` refuses all commands.
- [ ] A shell copy to the Desktop is refused before running.
- [ ] A redirect outside the workspace is refused before running.

- [ ] Quoted arguments with spaces stay one token.
- [ ] The child environment does not contain `OPENROUTER_API_KEY`.
- [ ] A timeout is handled gracefully.
- [ ] `safeEnv` keeps only allow-listed names.

If it goes wrong

- **The scan runs after the approval prompt** — move it before. Do not ask a human to approve a known-forbidden command.
- `ask` **becomes allow with no TTY** — an unanswerable prompt must deny.
- **The child sees the API key** — use `safeEnv`, not `process.env`.
- **The tokeniser is treated as a full shell parser** — it is a heuristic. Say so in the docs.

Checkpoint

1. Why is the hard deny-list checked before the policy?
2. Why is the command-line write scan done before the human approval prompt?
3. Why does `ask` degrade to `deny` when stdin is not a terminal?
4. What can still escape the sandbox even with all these layers?

Answers and fuller discussion: `course-manual.md` § Stage 9.

Stage 10 — Long-term memory

What you can do at the end: the agent remembers durable facts across runs and can search them on demand.

Plan: `plan.md` § Stage 10 · `course-manual.md` § Stage 10

Plan

1. Choose a file format a human can edit.
2. Parse forgivingly: bullets, optional dates, headings and prose ignored.
3. Implement the `Memory` class with bounds, deduplication, search and a bounded digest.
4. Inject a digest into the system prompt.
5. Add `remember` and `recall` tools.
6. Add `--memory` and `--no-memory` side doors.

Build

Files to create

- `src/memory.js`
- `src/memory.md` (initial store)
- `test/memory.test.js`

Files to modify

- `src/prompt.js` (inject the digest)
- `src/tools.js` (add `remember`, `recall`)
- `src/cli.js` (wire memory)

Format:

```
- [tag] text _(YYYY-MM-DD)_
```

Tag is one of `fact`, `pref`, `fix`, `gotcha`.

`Memory`:

- `new Memory({ root, file, maxEntries = 60, maxEntryChars = 300, rememberChars = 2000, promptChars = 1200, promptEntries = 12, now })`.
- `parseMemory`: accept `-` or `*`, optional date, normalise whitespace, ignore prose and headings.
- `remember(text, { tag })`: normalise; truncate; empty → error; duplicate refreshes the date; new appends; drop the oldest over `maxEntries`.
- `forget(match)`: delete entries whose text contains the match, case-insensitive.
- `search(query, { limit = 20 })`: AND over words; empty query returns all; newest first.
- `digest({ entries, chars })`: newest first; add while it fits; state the hidden count.
- `summary()`: `<label> (<n> memory/memories[, newest YYYY-MM-DD])`.

Tools:

- `remember`: `text` required, `tag` optional.
- No store → `ERROR: no memory store in this run (started with --no-memory)`.
- Empty → `ERROR: nothing to remember - the note was empty`.
- Duplicate → `already remembered... date refreshed`.
- New → `remembered...`.
- `recall`: `query` optional, `limit` default 20.
- No hits → `no memories in <label> match "<query>" (<size> stored)`.

CLI:

- `--memory` prints the store, the digest and the raw file, then exits 0.
- `--no-memory` disables memory for that run.
- `MEMORY_FILE` overrides the default `src/memory.md`.

Initial `src/memory.md`:

```
# memory

Long-term notes this agent keeps between runs. One bullet per memory:
`- [tag] text _(YYYY-MM-DD)_` where tag is fact | pref | fix | gotcha.

This file IS the store - edit it by hand, delete a wrong line, or commit it.
The agent reads a bounded slice at the start of a run and can search the rest
with the `recall` tool.

- [pref] The user's name is Craig. _(2026-10-28)_
- [pref] The user lives in Brighton, UK. _(2026-10-28)_
- [gotcha] The run_command shell in this sandbox is Windows cmd.exe, so Unix tools like `cat`, `ls` may fail; use
`type`, `dir` etc. instead. _(2026-10-28)_
- [pref] The user is a Python developer. _(2026-10-28)_
- [pref] The user has two dogs. _(2026-10-28)_
```

Run the incremental stage

```
node --test test/memory.test.js
node src/cli.js --memory
node src/cli.js "Remember that my favourite editor is Neovim"
node src/cli.js "What is my favourite editor?"
```

Expected: `--memory` prints the digest and exits 0; the remember task adds a bullet; a new process recalls it from the file.

Prove it

- [ ] Write and read back survive a new `Memory` instance.
- [ ] Duplicate refreshes the date; no duplicate bullet.
- [ ] `maxEntries` drops the oldest.
- [ ] Long memory is truncated to the per-entry cap.
- [ ] Empty/whitespace note is refused.

- [ ] The digest is bounded, newest first, and states the hidden count.
- [ ] Empty/disabled store adds and logs nothing.
- [ ] `forget` removes matches.
- [ ] Hand edits are picked up immediately.
- [ ] `remember` / `recall` work through tools.
- [ ] No memory store returns an error string, not a throw.
- [ ] `--memory` prints the digest and exits 0.
- [ ] `--no-memory` disables memory for that run.

If it goes wrong

- **Hand edits are ignored** — you cached entries. Read the file each time.
- **The digest grows without bound** — cap both entries and characters, and state what was hidden.
- **Tests write to the real** `src/memory.md` — use a temporary directory.
- **The agent remembers task progress** — memory is for durable facts, not a diary.

Checkpoint

1. Why does every `Memory` method read the file instead of using a cache?
2. Why does a duplicate refresh the date instead of appending?
3. What is the difference between the eager digest and lazy `recall`, and why have both?
4. What does `MEMORY_PROMPT_CHARS=0` change, and what does it leave working?

Answers and fuller discussion: `course-manual.md` § Stage 10.

Stage 11 — Lifecycle hooks

What you can do at the end: attach code to nine named moments in the loop, including four that can change the run.

Plan: `plan.md` § Stage 11 · `course-manual.md` § Stage 11

Plan

1. Define the nine events with exact names, icons, colours, timing and powers.
2. Implement the registry: `on`, `off`, `count`, `has`, `attached`, `run`.
3. Make four hooks decisive: `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`.
4. Decide the contract for notes, rewrites, replacements and blocks.
5. Make a thrown hook reported and ignored.
6. Generate the catalogue and trace from the same data the loop uses.

Build

Files to create

- `src/hooks.js`
- `test/hooks.test.js`

Files to modify

- `src/cli.js` (wire hooks into the manual tool loop)

The nine events:

#	NAME	ICON	COLOUR	WHEN	CAN
1	`SessionStart`	🚀	cyan	once, before the first model call	refuse, note
2	`UserPromptSubmit`	✍️	blue	once, as the task enters the loop	rewrite task, block
3	`PreModel`	🧠	magenta	before every model call	observe
4	`PostModel`	💬	cyan	after every reply	observe
5	`PreToolUse`	🛠️	yellowBright	before every tool call	block
6	`PostToolUse`	📦	green	after every tool call	replace observation
7	`Stop`	🏁	greenBright	the model answered, loop ending	observe
8	`StepLimit`	⏳	yellow	maxSteps reached	observe
9	`SessionEnd`	🔒	red	always, finished or not	observe

Return shapes:

- `{ note }`
- `{ task }` (submit only)
- `{ replace }` (post-tool only)

- `{ block: true, reason }`; the first block wins.

`Hooks`:

- `on(name, fn, { id, once })`
- `off(name, id)`
- `count(name)`
- `has(name)`
- `attached()`
- `run(name, payload)`

`traceHooks({ log, detail, explain })` prints one tracer per event:

```
  <icon>  [HOOK:<Name>] <when> - <detail>
```

`printHookCatalogue({ log, hooks })`; `HOOK_SUMMARY = { total: 9, deciding: 4, names }`.

CLI:

- `--hooks` prints the catalogue and exits 0.
- `--trace-hooks` is on by default.
- `--no-hooks` silences the trace.
- Fire all nine at the right moments.

Run the incremental stage

```
node --test test/hooks.test.js
node src/cli.js --hooks
node src/cli.js --no-hooks --demo
```

Expected: the catalogue lists all nine events and marks attached hooks; `--hooks` works with no key; `--no-hooks` runs the demo without hook traces.

Prove it

- [ ] Every hook has icon, colour, when, gets, may and why.
- [ ] The catalogue prints all 9 and marks attached hooks.
- [ ] An unknown hook name throws.
- [ ] Handlers run in order; the first block wins.
- [ ] A throwing hook is reported, not fatal.
- [ ] Notes print; `once` handlers detach.
- [ ] `off` by id works.
- [ ] Trace prints `[HOOK:Name]` by default.
- [ ] `--no-hooks` silences the trace.
- [ ] `--hooks` prints the catalogue with no key.

If it goes wrong

- **Hooks are notifications only** — the four deciding hooks must return a decision the loop honours.
- **A later hook unblocks** — the first block wins.
- **A thrown hook kills the loop** — catch it, report it, continue.
- **The catalogue and the loop drift** — generate both from one event table.

Checkpoint

1. Why are there exactly nine hooks, and how were they chosen?
2. Which four hooks can change the run, and what can each change?
3. Why does a hook that throws not stop the loop?
4. Why does the first block win?

Answers and fuller discussion: `course-manual.md` § Stage 11.

Stage 12 — The ReAct loop (`agent.js`)

What you can do at the end: run the whole agent as a testable library, with a step limit, a truncation guard and a heartbeat.

Plan: `plan.md` § Stage 12 · `course-manual.md` § Stage 12

Plan

1. Extract the manual loop from `cli.js` into `runTask`.
2. Return outcomes rather than throwing task results.
3. Rebuild the system prompt for every task.
4. Implement the seven beats per step.
5. Add the truncation guard.
6. Add the heartbeat for slow model calls.
7. Guard direct execution of `agent.js`.

Build

Files to create

- `src/agent.js`
- `src/heartbeat.js`
- `test/agent.test.js`

Files to modify

- `src/cli.js` (replace the manual loop with `runTask`)

`runTask` signature:

```
runTask({
  task,
  workspace,
  sandbox = null,
  commandPolicy,
  maxSteps,
  chatFn,
  confirm,
  hooks = null,
  memory = null,
  history = [],
  log = console.log,
  heartbeatMs = 5000
})
```

Return `{ ok, answer, messages, steps }` or `{ ok: false, reason, messages }`.

Implementation:

- Rebuild the system prompt every task; filter old system messages out of `history`.

- `ctx = { workspace, sandbox, commandPolicy, memory, confirm }`; `confirm` defaults to deny.
- `hooks ?? new Hooks()` so every call site can call `hook.run` unconditionally.
- Direct execution prints the "agent.js is the loop, not the program" hint; never import `cli.js`.

Seven beats per step:

1. Blank line and `ui.step`.
2. Fire `PreModel`.
3. Start the heartbeat; call `chatFn`; on throw stop the heartbeat, print the error, fire `SessionEnd`, return `request_failed`; otherwise stop the heartbeat.
4. Extract the choice and message; fire `PostModel`.
5. Truncation guard: `finish_reason === 'length'` with no tool calls → append the assistant, append a user message `"You were cut off by the length limit. Continue with smaller tool calls."`, warn, `continue`. Never dispatch truncated calls.
6. Append `sanitizeAssistant`; print `ui.think` if there is content.
7. No calls → print the answer, fire `Stop` and `SessionEnd`, return `ok`. Calls → for each: print the tool, fire `PreToolUse`, block or run, print the result, fire `PostToolUse`, append exactly one tool message.

After `maxSteps`: print an error, fire `StepLimit` then `SessionEnd`, return `max_steps`.

Entry guards:

- `SessionStart` or `UserPromptSubmit` blocked → return `blocked_by_hook`.
- A rewrite replaces `messages[taskIndex]`.

`heartbeat(log, everyMs)` returns `{ stop }`; it prints `… still waiting for the model (<s>s)` and calls `timer.unref?.()`.

Run the incremental stage

```
node --test test/agent.test.js
node src/agent.js
node src/cli.js "Run node --version and tell me the result"
```

Expected: the agent test suite runs the loop with a fake model and no network; `node src/agent.js` prints the hint; the live task still works through the new library.

Prove it

- [ ] Tools run, then the loop stops on a plain answer.
- [ ] A failing tool result is fed back so the model can correct.
- [ ] The loop is capped at `maxSteps`.
- [ ] A truncated reply is not dispatched; the model is asked to continue smaller.
- [ ] The heartbeat prints waiting lines during slow calls.
- [ ] Blocked hooks produce the right exit path.
- [ ] The step limit fires `StepLimit` then `SessionEnd`.

- [ ] A follow-up task sees previous conversation but a fresh system prompt.
- [ ] Direct execution prints the hint and does not import the CLI.

If it goes wrong

- **The system prompt appears twice** — filter old system messages from `history` and rebuild once.
- **A truncated call is dispatched** — check `finish_reason` before dispatching.
- **The process stays alive after a run** — `unref` the heartbeat timer.
- **The loop throws task outcomes** — return a reason; the CLI maps it to an exit code.

Checkpoint

1. Why is `runTask` a library rather than part of `cli.js`?
2. Why are `request_failed` and `max_steps` returned rather than thrown?
3. Walk through the seven beats from memory. Which beat fires which hook?
4. Why is a truncated tool call not dispatched?

Answers and fuller discussion: `course-manual.md` § Stage 12.

Stage 13 — Offline demo

What you can do at the end: prove the safety guards with a scripted model and a scorecard, with no network and no key.

Plan: `plan.md` § Stage 13 · `course-manual.md` § Stage 13

Plan

1. Write a fake model that walks a fixed script.
2. Script 14 tool steps plus a final answer, including deliberate escapes and denials.
3. Keep the loop, tools, sandbox, hooks and memory real; replace only the HTTP client.
4. Build a scorecard from the actual tool observations.
5. Exit 0 only on 7/7 checks.

Build

Files to create

- `src/demo.js`

Files to modify

- `src/cli.js` (`--demo`)
- `test/agent.test.js` (demo assertions)

`demoChat()` returns an async `chatFn` walking `DEMO_SCRIPT`.

Tool call shape:

```
{
  id,
  type: 'function',
  function: { name, arguments: JSON.stringify(args) }
}
```

The 14 steps:

1. `list_files` `{ path: '.' }`
2. `create_file` `{ path: 'demo/hello.js', content: ... }`
3. `run_command` `{ command: 'node demo/hello.js' }`
4. `edit_file` on the greeting
5. `run_command` again
6. `write_file` `../escape.txt` → expect `FORBIDDEN`
7. `read_file` `../package.json` → allowed
8. `write_file` `~/Desktop/pwned.txt` → expect `FORBIDDEN`
9. `run_command` shell copy to Desktop → expect `FORBIDDEN`
10. `run_command` `git push --force` → expect `DENIED`

11. `run_command` that echoes `OPENROUTER_API_KEY` → expect `absent`
12. `run_command` that exits 1 → expect `exit code 1`
13. `remember`
14. `recall`

Each assistant step has a short natural-language `content` before its tool call. `maxSteps = DEMO_SCRIPT.length + 1 = 15`.

`sandboxScorecard(messages)` maps `tool_call_id` to its result and regex-checks:

- `s6` `/^FORBIDDEN: SAVE \.\.\/escape\.txt/`
- `s7` `/^\.\.\/package\.json \(\d+ lines\)/`
- `s8` `/^FORBIDDEN: SAVE ~\/Desktop\/pwned\.txt/`
- `s9` `/^FORBIDDEN: COMMAND COPY /`
- `s10` `/^DENIED: blocked by policy/`
- `s11` `/OPENROUTER_API_KEY=absent/`
- `s12` `/^exit code 1\b/`

CLI: `--demo` uses the scripted chat, `maxSteps = 15`, auto-approves commands, prints the scorecard, exits 0 on 7/7 and `process.exitCode = 1` otherwise.

Run the incremental stage

```
node src/cli.js --demo
echo "exit=$?"
node src/cli.js --demo
```

Expected: 14 tool steps plus a final answer; `7/7 checks held`; exit 0. On the second run, `create_file` refuses to clobber and the run still finishes cleanly.

Prove it

- [ ] `node src/cli.js --demo` completes 14 steps + a final answer.
- [ ] It creates, edits and runs `workspace/demo/hello.js`.
- [ ] It refuses `../escape.txt`, `~/Desktop/pwned.txt` and the shell copy.
- [ ] It allows the read of `../package.json`.
- [ ] It denies `git push --force`.
- [ ] The child prints `OPENROUTER_API_KEY=absent`.
- [ ] It reports exit code 1 with the explanation.
- [ ] It remembers and recalls.
- [ ] The scorecard prints **7/7 checks held**.
- [ ] The process exits 0.

If it goes wrong

- **The demo stops one step early** — `maxSteps` must be 15, not 14.

- **The demo uses a fake sandbox** — replace only the model client. The guards must be real.
- **A red row is treated as a UI bug** — it is a regression. Fix the guard.
- **The second run fails on** `create_file` — that is expected; the model should recover.

Checkpoint

1. What does `demoChat()` replace, and what does it leave real?
2. Why can the demo test guards that a live model might not exercise?
3. Why does `--demo` need a `maxSteps` of 15?
4. What happens on a second demo run, and why is that instructive?

Answers and fuller discussion: `course-manual.md` § Stage 13.

Stage 14 — Sessions, trimming and CLI polish

What you can do at the end: run multiple tasks in one process with bounded history, a reproducible banner and meaningful exit codes.

Plan: `plan.md` § Stage 14 · `course-manual.md` § Stage 14

Plan

1. Add `trimHistory` with a safe `user` boundary.
2. Run a session loop that carries trimmed history between tasks.
3. Give every task a fresh step budget.
4. Print a banner with provenance before the first task.
5. Add warnings for a missing `/` in the model name and for normalisation.
6. Map outcomes to exit codes `0`, `1`, `2`, `3`.
7. Call `main()` only when `cli.js` is the entry file.

Build

Files to modify

- `src/cli.js` (major polish)
- `test/session.test.js`

Implementation:

- `MAX_HISTORY = 24`.
- `trimHistory(messages, max)`:
- drop system messages;
- keep at most `max`;
- move `start` forward until it lands on a `user` message;
- if there is no safe boundary, return `[]`.
- `runSession`:
- `askTask` → command or task;
- `runTask({ ...base, task, history })`;
- `history = trimHistory(result.messages)`.
- Every task gets a fresh step budget.
- After each task: divider, pass/fail, message-count hint, retry hint.
- On exit: `session over - N task(s) run.`
- Banner before step 1: workspace, sandbox, model and source, key fingerprint and source, env file, command policy, timeout, tools, memory summary, hooks state.
- Warnings: live mode and the model lacks `/`; a normalised bare name note.
- Exit codes: `0` normal; `1` request/fatal; `2` no task/no TTY; `3` step limit.
- `taskExitCode(result)` maps the result as above.

- `main()` catches fatal errors, prints `fatal: <stack>`, sets `process.exitCode`.
- Only call `main()` when `import.meta.url === pathToFileURL(process.argv[1]).href`.

Run the incremental stage

```
node --test test/session.test.js
node src/cli.js
node src/cli.js --max-steps 1 "Keep calling a tool forever"
echo "exit=$?"
```

Expected: a follow-up sees the previous conversation; `/new` resets history but not workspace or memory; the step-limit task exits 3; no task and no TTY exits 2.

Prove it

- [ ] `/new` clears conversation history but leaves workspace and memory untouched.
- [ ] `/quit` and `/exit` end the session.
- [ ] A follow-up sees previous conversation but a fresh system prompt with the latest memory.
- [ ] Every task starts its own step budget.
- [ ] A task-rewriting hook rewrites the new task, not history.
- [ ] No task + non-TTY exits 2.
- [ ] Step limit exits 3.
- [ ] Live mode without a key exits 1.
- [ ] The banner prints on every run.
- [ ] The exit note explains non-zero codes.

If it goes wrong

- **History starts with a `tool` message** — move the boundary forward to a `user` message.
- **The system prompt is carried in history** — filter it out and rebuild.
- **The step limit does not exit 3** — map `reason: 'max_steps'` in `taskExitCode`.
- **Tests run `main()` when they import `cli.js`** — use the entry guard.

Checkpoint

1. What is the difference between a task and a session?
2. Why does the step budget restart for each task?
3. Why must `trimHistory` start on a `user` message?
4. What do exit codes `1`, `2` and `3` each mean?

Answers and fuller discussion: `course-manual.md` § Stage 14.

Stage 15 — The full test suite

What you can do at the end: run `npm test` fully offline and trust it.

Plan: `plan.md` § Stage 15 · `course-manual.md` § Stage 15

Plan

1. Mirror the module graph with one suite per module.
2. Test the model client with a mocked `fetch`.
3. Test the loop with a fake model.
4. Test the truncation guard and the hooks.
5. Test memory with a temporary directory.
6. Make every suite offline and independent.

Build

Files to create or complete

- `test/workspace.test.js` — path policy, containment, symlink safety.
- `test/sandbox.test.js` — refusals, root deletion, tool-layer conversion.
- `test/tools.test.js` — all eight tools, validation, `safeEnv`, policy.
- `test/shell-sandbox.test.js` — tokenisation, write-target scanning.
- `test/agent.test.js` — loop behaviour, truncation, heartbeat, demo guards.
- `test/hooks.test.js` — registry, blocking, replacing, tracing, catalogue.
- `test/memory.test.js` — round-trip, duplicates, bounds, digest, forget, search.
- `test/prompt.test.js` — rules, injection, logging.
- `test/llm.test.js` — timeout, errors, sanitisation.
- `test/config.test.js` — `.env` precedence, fingerprint, model normalisation.
- `test/session.test.js` — commands, trimming, follow-up tasks.
- `test/ui.test.js` — styling, verbatim output, result classification.
- `test/input.test.js` — `askTask`, blank lines, Ctrl-C.

Test rules:

- Mock `fetch`; assert on the request; restore in `finally`.
- Use a temporary directory for memory.
- Do not assert raw ANSI without controlling the colour level.
- Test the loop, not the model. A fake `chatFn` is enough.
- A failing tool result must be fed back so the model can recover.

Run the incremental stage

```
npm test
```

Expected: all suites pass with no network and no API key. Unplug the network or set an invalid key and run it again; the result should not change.

Prove it

- [ ] `npm test` runs fully offline.
- [ ] All suites pass.
- [ ] No real network calls.
- [ ] No API key required.
- [ ] Each module has a suite that fails when that module breaks.

If it goes wrong

- **A mocked `fetch` leaks** — restore it in `finally`.
- **Tests write to the real memory file** — use `mkdtemp` and clean up.
- **A test needs a live model** — the seam is wrong; inject a fake `chatFn`.
- **A test fails only on another machine** — you asserted ANSI codes or a platform-specific path separator.

Checkpoint

1. Why does the test suite mirror the module graph?
2. How do you mock `fetch` safely?
3. Why must the loop be testable without a model?
4. Why must memory tests use a temporary directory?

Answers and fuller discussion: `course-manual.md` § Stage 15.

Stage 16 — Documentation and final acceptance

What you can do at the end: hand the project to another reader and prove all twenty acceptance checks on a clean checkout.

Plan: `plan.md` § Stage 16 · `course-manual.md` § Stage 16

Plan

1. Write a README that teaches the architecture, not just the commands.
2. Document the loop, the tools, the hooks, memory, sessions, configuration and exit codes.
3. State the safety model **and its limits**.
4. Add `demo_prompt.md` as a repeatable demonstration and acceptance script.
5. Run the twenty final checks on a clean checkout.

Build

Files to create

- `README.md`
- `demo_prompt.md`
- `.env.example` (if absent)

The README must contain:

- a one-paragraph summary;
- quick start: `npm install`, copy `.env.example` to `.env`, set the key, `npm start`, `npm run demo`;
- the loop diagram and the seven beats;
- a file-by-file tour, `agent.js` first;
- the eight tools and what each is for;
- the read/write asymmetry and the shell limitation;
- the nine lifecycle hooks, with a table;
- long-term memory: file format, eager digest vs lazy recall, `--memory`;
- sessions and history trimming;
- a configuration table with defaults and precedence;
- exit codes;
- the safety model: workspace fence, hard denies, environment allow-list, command policy, and an explicit **"this is not a container"** statement;
- examples: a Desktop write that ends in `FORBIDDEN`; a prompt that ends with `remember`.

`demo_prompt.md` should contain prompts such as:

```
create hello.js that prints today's date and time, run it, and save its output to hello-output.txt
save a file test.txt that contains 'HELLO WORLD' and save to the desktop
npm run demo
node src/cli.js --memory
work out how this package is tested, run the tests, and remember the command
```

Run the incremental stage

```
npm install
npm test
node src/cli.js --help
node src/cli.js --hooks
node src/cli.js --memory
node src/cli.js --demo
```

Expected: install succeeds with only `chalk`; tests pass offline; help, hooks and memory work with no key; the demo reports **7/7 checks held** and exits 0.

Prove it — the twenty final checks

- [ ] 1. `npm install` succeeds with only `chalk` as a runtime dependency.
- [ ] 2. `npm test` runs fully offline and all suites pass.
- [ ] 3. `node src/cli.js --help` prints usage without a key.
- [ ] 4. `node src/cli.js --hooks` prints the full nine-hook catalogue without a key.
- [ ] 5. `node src/cli.js --memory` prints the store and the exact digest.
- [ ] 6. `node src/cli.js --demo` completes the script with **7/7 checks held**.
- [ ] 7. Live mode with a valid key runs a task through OpenRouter.
- [ ] 8. Live mode without a key prints the missing-key error and exits 1.
- [ ] 9. No task + no TTY exits 2.
- [ ] 10. A task that never yields an answer stops at the step limit and exits 3.
- [ ] 11. A write outside `workspace/` prints one `⛔ FORBIDDEN` line and returns a `FORBIDDEN:` observation without touching disk.
- [ ] 12. A read outside `workspace/` succeeds.
- [ ] 13. `CMD_POLICY=deny` refuses all commands; hard-denied commands are refused even with `--yes`.
- [ ] 14. The child shell cannot read `OPENROUTER_API_KEY`.
- [ ] 15. Memory survives process restarts; duplicates refresh; the file is bounded; hand edits are picked up.
- [ ] 16. Every normal run prints `[HOOK:Name]` traces; `--no-hooks` silences them.
- [ ] 17. `/new` clears conversation history but leaves workspace and memory untouched; `/quit` and `/exit` end the session.
- [ ] 18. A follow-up sees previous conversation but a freshly rebuilt system prompt with the latest memory.
- [ ] 19. `agent.js` executed directly prints the hint and does not run the CLI.
- [ ] 20. No framework, no agent SDK, no database, no hidden global state.

If it goes wrong

- **The README is only commands** — add the loop, the file tour and the safety limits.
- **The docs claim the sandbox is secure** — state that it is a teaching policy, not a container.
- `demo_prompt.md` **no longer matches behaviour** — treat it as an acceptance test and update it with the code.
- **A final check fails on a clean checkout** — fix the setup, not the check.

Checkpoint

1. Why is the README part of the architecture?
2. Why must the docs state the sandbox's limits?
3. What makes a good demo prompt?
4. Pick three final acceptance checks and explain which design decision makes each one pass.

Answers and fuller discussion: `course-manual.md` § Stage 16.

Part III: Finishing

The course checklist

Copy this into your notes and tick as you go.

STAGE	MODULE	WHAT YOU CAN DO AFTER THIS STAGE	ACCEPTED
0	Scaffold	`npm install` works	☐
1	`ui.js`	Print styled terminal output	☐
2	`config.js` + `llm.js`	Load `.env`, call OpenRouter	☐
3	Minimal `cli.js`	**Chat with the model** 🎉	☐
4	History in `cli.js`	Have a **multi-turn conversation**	☐
5	`prompt.js`	Inject behavioural rules	☐
6	`workspace.js`	Fence writes, open reads	☐
7	Read tools	Model **lists and reads files**	☐
8	Sandbox + write tools	Model **creates and edits files** safely	☐
9	`run_command`	Model **runs shell commands** with policy	☐
10	`memory.js`	Model **remembers and recalls** across runs	☐
11	`hooks.js`	Observe/rewrite/block lifecycle events	☐
12	`agent.js`	Extract the loop into a **testable library**	☐
13	`demo.js`	**Offline demo** proves all guards hold	☐
14	Full CLI polish	**Sessions**, history trimming, banner, exit codes	☐
15	Tests	`npm test` passes fully offline	☐
16	Docs	README teaches the architecture	☐

Where to go next

The workbook ends with a working harness. The manual continues with:

- **Part II — the teacher's handbook:** pacing, run sheets, live-demo scripts, the factory analogy, common misconceptions and an assessment rubric.
- **Part III — the student workbook:** per-stage labs, break-it labs, capstones, extension roadmap and self-assessment.
- **Part IV — deep dives:** context engineering, tool design, evaluation, threat modelling and the production path.
- **Part V — reference:** configuration, CLI, payloads, file and API map, and the final acceptance checklist.

Useful next builds:

- add a `search_files` or `run_tests` tool with the same schema/validation/string-result discipline;
- add a `PostToolUse` redactor for secrets;
- replace the file-backed memory with a small database and compare the cost and auditability;
- add streaming and measure what changes in the UI and the loop;
- write a real evaluation suite with recorded transcripts, not just unit tests.

Troubleshooting by symptom

SYMPTOM	LIKELY CAUSE	FIRST FIX
`Cannot use import statement outside a module`	Missing `"type": "module"`	Set it in `package.json`
`OPENROUTER_API_KEY is not set`	No `.env` or no key in it	Copy `.env.example` to `.env`
`401` from OpenRouter	Bad or revoked key	Rotate the key; never print it
`402` from OpenRouter	No credits	Top up or use the offline demo
The program hangs at a prompt	No TTY and waiting for stdin	Check `process.stdin.isTTY`; exit 2
The model calls a tool that does not exist	The prompt names tools before they are built	Pass the real tool list only when the tools exist
A write outside `workspace/` succeeds	`startsWith` containment or no symlink resolution	Use resolved path boundaries
A shell copy writes outside the workspace	The command scan is missing a shape	Add the command family and a test
The child process sees the API key	`process.env` was passed to `exec`	Use `safeEnv`
History starts with a `tool` message	Arbitrary trimming	Move to a `user` boundary
`npm test` needs the network	A real `fetch` or `chat` is in a test	Inject a fake or mock `fetch`
The demo reports fewer than 7/7	A guard regressed	Fix the guard, not the scorecard

Glossary and reference

The full glossary, message protocol, safety model, bounds, debugging guide and configuration reference are in `course-manual.md` Appendices and Part V. The concise build contract for every stage is in `plan.md`.

You built a harness. Not a product, not a framework, not a container. A program whose loop, tools, safety layers and extension points you can explain line by line. That is the transferable skill. Every coding agent you meet is now a variation on something you have already built.